actually changes the DOM, and that the changes are sent back to the server accurately.

I want to split the integration test for adding a task into one spec that adds the task, and one test that handles the reordering. This isolates the JavaScript to just the task that handles reordering:

js_integration/01/spec/system/add_task_spec.rb

```
require "rails_helper"

RSpec.describe "adding a new task" do
  let!(:project) { create(:project, name: "Project Bluebook") }
  let!(:task_1) { create(
    :task, project: project, title: "Search Sky", size: 1, project_order: 1) }
  let!(:task_2) { create(
    :task, project: project, title: "Use Telescope", size: 1,
           project_order: 2) }
  let!(:task_3) { create(
    :task, project: project, title: "Take Notes", size: 1,
           project_order: 3) }

  it "can add a task" do
    visit(project_path(project))
    fill_in("Task", with: "Find UFOs")
    select("2", from: "Size")
    click_on("Add Task")
    expect(current_path).to eq(project_path(project))
    within("#task_4") do
      expect(page).to have_selector(".name", text: "Find UFOs")
      expect(page).to have_selector(".size", text: "2")
    end
  end

  it "can re-order a task", :js do
    visit(project_path(project))
    within("#task_3") do
      click_on("Up")
    end
    expect(page).to have_selector(
      "tbody:nth-child(2) .name", text: "Take Notes")
    visit(project_path(project))
    within("#task_2") do
      expect(page).to have_selector(".name", text: "Take Notes")
    end
  end

end
```

I made some relatively minor changes. I added a task to the setup so that you'd have three tasks in the reorder spec. The reorder spec does the reorder, checks to see that it works, then reloads the page and checks to see that the